

FastNoise SIMD Unity Documentation

FastNoise SIMD Unity should be used when you want a Unity Editor component to edit values of a FastNoise SIMD object.

To change FastNoise SIMD settings through script access the FastNoise SIMD object through the FastNoise SIMD Unity component, for example:

```
myFastNoiseSIMDUnityComponent.fastNoiseSIMD.SetSeed(10)
```

Settings can be changed in the FastNoise SIMD Unity public variables, but these changes will not be applied unless `SaveSettings()` is called.

FastNoise SIMD Documentation

Getting Started

Since SIMD functions work by processing multiple values at once this library functions by returning sets of data rather than single values. A set is returned as an array of floats.

Example of how to generate a set of data and process it.

```
FastNoiseSIMD myNoise = new FastNoiseSIMD();

// Get a set of 16 x 16 x 16 Simplex Fractal noise
float[] noiseSet = myNoise.GetNoiseSet(0, 0, 0, 16, 16, 16);
int index = 0;

for (int x = 0; x < 16; x++)
{
    for (int y = 0; y < 16; y++)
    {
        for (int z = 0; z < 16; z++)
        {
            // Pseudo function to process data in noise set
            ProcessVoxelData(x, y, z, noiseSet[index++]);
        }
    }
}
```

Note: If the size of the Z dimension is a multiple of 8 there is a slight performance boost

Noise Generation

```
float[] GetNoiseSet(float x, float y, float z, int xSize, int ySize, int zSize, float scaleModifier = 1.0f)
```

Returns a set of noise using noise type as set by `SetNoiseType()`.

```
void FillNoiseSet(float[] noiseSet, int xStart, int yStart, int zStart, int xSize, int ySize, int zSize, float scaleModifier = 1.0f)
```

Fills the given noise set using noise type as set by `SetNoiseType()`, the `noiseSet` array must be initialised and have a size equal or larger to `xSize*ySize*zSize`.

```
float[] GetSampledNoiseSet(int xStart, int yStart, int zStart, int xSize, int ySize, int zSize, int sampleScale)
```

Returns a set of sampled noise using noise type as set by `SetNoiseType()`.

Sampled noise calculates a reduced number of noise points and then interpolates the data to fill the noise set, this provides a large performance increase for a small drop in fidelity. A higher `sampleScale` means a smaller sample size and more interpolation, a `sampleScale` of 1-2 is recommended.

```
void FillSampledNoiseSet(float[] noiseSet, int xStart, int yStart, int zStart, int xSize, int ySize, int zSize, int sampleScale)
```

Fills the given noise set using sampled noise of type as set by `SetNoiseType()`, the `noiseSet` array must be initialised and have a size equal or larger to `xSize*ySize*zSize`.

Sampled noise calculates a reduced number of noise points and then interpolates the data to fill the noise set, this provides a large performance increase for a small drop in fidelity. A higher `sampleScale` means a smaller sample size and more interpolation, a `sampleScale` of 1-2 is recommended.

Noise Settings

General

```
int GetSeed()
```

Returns seed

```
void SetSeed(int seed)
```

Default: 1337

Using different seeds will cause the noise output to change

Used in all noise generation

```
void SetFrequency(float frequency)
```

Default: 0.01

Affects how coarse the noise output is

Used in all noise generation except White Noise

```
void SetNoiseType(NoiseType noiseType)
```

Default: Simplex

Sets the type of noise returned by GetNoise()

Possible noise types:

- Value
- ValueFractal
- Perlin
- PerlinFractal
- Simplex
- SimplexFractal
- Cellular
- WhiteNoise
- Cubic
- CubicFractal

Fractal

```
void SetFractalOctaves(int octaves)
```

Default: 3

The amount of noise layers used to create the fractal

Used in all fractal noise generation

```
void SetFractalLacunarity(float lacunarity)
```

Default: 2.0

The frequency multiplier between each octave

Used in all fractal noise generation

```
void SetFractalGain(float gain)
```

Default: 0.5

The relative strength of noise from each layer when compared to the last

Used in all fractal noise generation

```
void SetFractalType(FractalType fractalType)
```

Default: FBM

Used in all fractal noise generation

Possible fractal types:

- FBM
- Billow

- RigidMulti

Cellular

```
void SetCellularDistanceFunction(CellularDistanceFunction cellularDistanceFunction)
```

Default: Euclidean

The distance function used to calculate the cell for a given point

Possible distance functions:

- Euclidean
- Manhattan
- Natural

(Natural is a blend of Euclidean and Manhattan to give curved cell boundaries)

```
void SetCellularReturnType(CellularReturnType cellularReturnType)
```

Default: Cell Value

What value does the cellular function return from its calculations

Possible return types:

- CellValue
- Distance
- Distance2
- Distance2Add
- Distance2Sub
- Distance2Mul
- Distance2Div
- NoiseLookup
- Distance2Cave

Only CellValue is bounded between -1.0 and 1.0, all distance return types have a minimum of 0 and varying maximums.

```
void SetCellularNoiseLookupType(NoiseType cellularNoiseLookupType)
```

Default: Simplex

Sets the type of noise used if cellular return type is set the NoiseLookup

Possible noise types:

- Value
- ValueFractal
- Perlin
- PerlinFractal
- Simplex
- SimplexFractal

```
void SetCellularNoiseLookupFrequency(float cellularNoiseLookupFrequency)
```

Default: 0.2

Sets relative frequency on the cellular noise lookup return type

```
void SetCellularDistance2Indicies(int cellularDistanceIndex0, int  
cellularDistanceIndex1)
```

Default: 0, 1

Sets the 2 distance indicies used for distance2 return types

Note: index0 should be lower than index1

Both indicies must be ≥ 0 , index1 must be < 4

```
void SetCellularJitter(float cellularJitter)
```

Default: 0.45

Sets the maximum distance a cellular point can move from its grid position

Note: Setting this high will make cellular grid artifacts more common

Perturb

```
void SetPerturbType(PerturbType perturbType)
```

Default: None

Enabling perturbing affects all noise types, it warps the input coordinates in 3D space before generating noise.

Possible types:

- None
- Gradient
- GradientFractal
- Normalise
- Gradient_Normalise
- GradientFractal_Normalise

The normalise types will normalise the input position then scale it by the value `SetPerturbNormaliseLength()`, this is useful for creating height maps for a sphere if you set the normalise length to the radius.

For `Gradient{Fractal}_Normalise` the gradient perturb step is done first then the result is normalised as explained above.

```
void SetPerturbAmp(float perturbAmp)
```

Default: 1.0

Sets the maximum distance the input position can be perturbed.

```
void SetPerturbFrequency(float perturbFrequency)
```

Default: 0.5

Sets the frequency of the perturb noise function, this is relative to the normal frequency `SetFrequency(float)`

```
void SetPerturbNormaliseLength(float perturbNormaliseLength)  
Default: 1.0
```

Sets the length for vectors after perturb normalising.

Following functions only apply to fractal perturb types

```
void SetPerturbFractalOctaves(int perturbOctaves)  
Default: 3
```

Sets octave count for perturb fractal types

```
void SetPerturbFractalLacunarity(float perturbLacunarity)  
Default: 2.0
```

Sets octave lacunarity for perturb fractal types

```
void SetPerturbFractalGain(float perturbGain)  
Default: 0.5
```

Sets octave gain for perturb fractal types

Vector Sets

Vector sets are an advanced feature of FastNoise SIMD that allow custom coordinates to be passed in for noise generation instead of the axis aligned grids that the normal functions will return. For example this could be used to calculate noise at points on the surface of a planet, they can also be used to circumvent the normal limitation of `zSize >= 4`.

```
new VectorSet(Vector3[] vectors, int samplingScale = 0)
```

Creates a new vector set using the provided array of `Vector3`. The `samplingScale` variable only applies when using `FillSampledNoiseSetVector()` and is used to store the sampling scale of the vector set.

```
void FillNoiseSetVector(float[] noiseSet, VectorSet vectorSet, float xOffset = 0f, float  
yOffset = 0f, float zOffset = 0f)
```

Fill the noise set with type set by `SetNoiseType()` using the positions in the vector set. The size of the noise set must be equal or larger than the vector set. x/y/z offsets can be used to offset every position in the vector set when calculating noise, this can be useful when reusing the same vector set for multiple world positions.

```
void FillSampledNoiseSetVector(float[] noiseSet, VectorSet vectorSet, float xOffset = 0f, float yOffset = 0f, float zOffset = 0f)
```

Same as above, FillNoiseSetVector(). The vector set points are used as the initial noise sampling locations and the returned noise values will be between these points, keep this in mind when creating sampling vector sets. The sampling scale used will be the value set in the vector set. The vector set positions will be in the same order as a noise set is normally returned, ordered zyx axis. The noise set provided must be equal or larger than the sampled vector set, so 8x for sampling scale 1 and 64x for 2.

SIMD Level

```
static int GetSIMDLevel()
```

Returns highest detected level of CPU support for SIMD

- 5: ARM NEON (Mobile)
- 4: AVX512
- 3: AVX2 & FMA3
- 2: SSE4.1
- 1: SSE2
- 0: Fallback, no SIMD support
- -1: Not initialised, create FastNoise SIMD object
- -2: No FastNoise SIMD support on this platform

Notes

- When compiling for Android only armv7 architecture is currently supported. Mono2x scripting backend is recommended
- When compiling for iOS only device SDK is supported, not simulator SDK. IL2CPP scripting backend is recommended